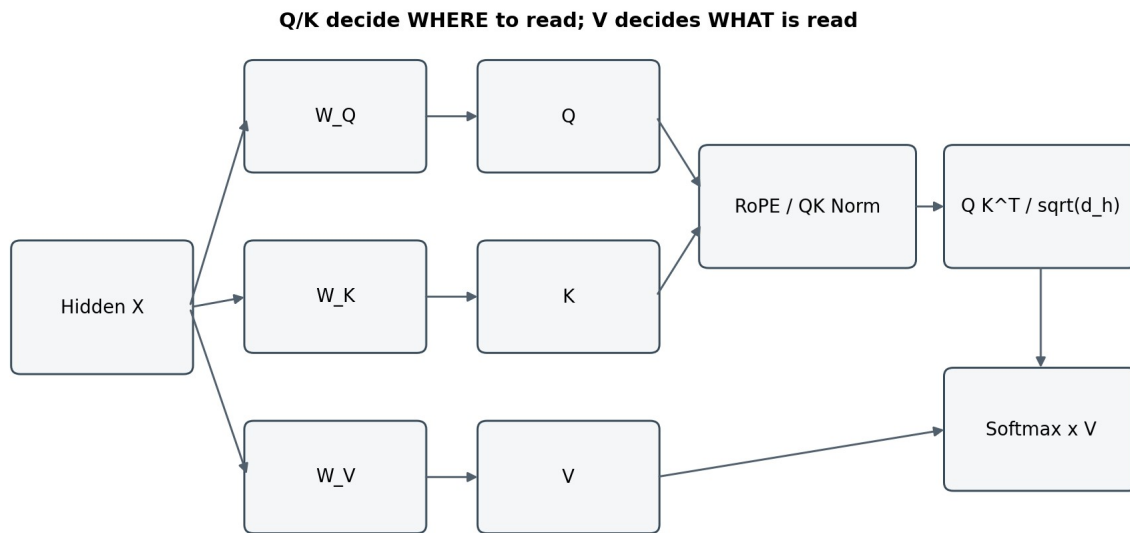


Q / K / V Projection

Query-Key-Value 投影与 Attention 信息路由

数学本质 · MHA/GQA/MQA · QK-Norm · RoPE 接口 · KV Cache · Tensor Parallel · MLA · Gotchas · Know-how · Know-why



The three projections are learned interfaces between the residual stream and the attention operator.

Research edition · August 2026

核心命题 Attention 不只是一个 softmax 公式。Q/K/V projection 决定 residual stream 以什么“查询语言”、什么“索引语言”、什么“内容语言”进入注意力算子；它同时是算法设计、训练稳定性和推理带宽的交叉点。

目录与阅读路径

1. Executive Summary: 12 个必须先记住的结论
2. Q/K/V 位于 Transformer 的哪里
3. 数学基础: 三次线性投影究竟做了什么
4. Know-why: 为什么要区分 Query / Key / Value
5. Shape、参数量、FLOPs 与初始化
6. Multi-Head Attention: head 不是简单切片
7. MHA $\rightarrow$ MQA $\rightarrow$ GQA: KV 共享为什么能加速
8. Cross-Attention: Q 与 K/V 来自不同信息源
9. Position Encoding 与 Q/K: 为什么 RoPE 通常不作用于 V
10. Attention Scaling、Softmax Saturation 与 QK-Norm
11. Fused QKV、Bias 与工程布局
12. 训练动力学: 梯度、范数、熵与 head collapse
13. KV Cache: 从 projection 直接推导推理显存
14. Tensor Parallel / Sequence Parallel / Context Parallel
15. FlashAttention 与 QKV: 算法相同、执行路径不同
16. Quantization / FP8 / Low Precision 的 QKV 风险
17. MLA: 从显式 K/V 到 latent KV
18. 45 个 Gotchas
19. Know-how: 从零实现与生产 Checklist
20. Know-why: 因果原理总表
21. Debugging / Monitoring 指标
22. Ablation Matrix: 如何验证设计选择
23. PyTorch MVP 与 Parity Tests
24. 2025-2026 新研究与开放问题
25. 面试与系统设计题
26. 参考文献

推荐阅读顺序 如果目标是“从零实现”，优先读 2 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 9 $\rightarrow$ 10 $\rightarrow$ 23；如果目标是“大模型推理”，优先读 7 $\rightarrow$ 13 $\rightarrow$ 14 $\rightarrow$ 15 $\rightarrow$ 16 $\rightarrow$ 17；如果目标是“训练稳定性”，优先读 10 $\rightarrow$ 12 $\rightarrow$ 18 $\rightarrow$ 21 $\rightarrow$ 22。

1. Executive Summary : 12 个必须先记住的结论

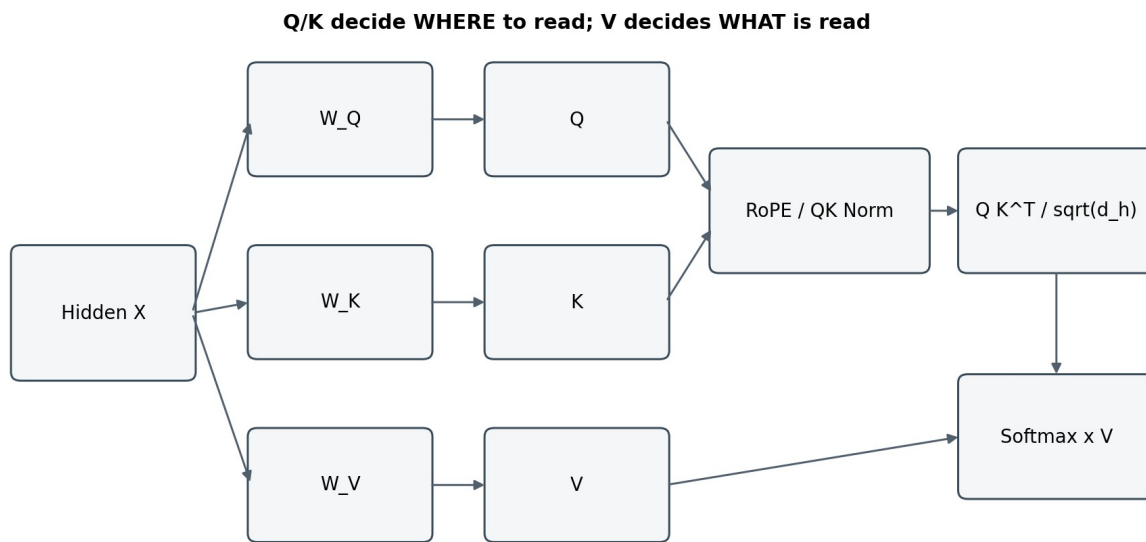
#	结论	Know-why
1	Q/K 决定匹配关系	单 head 中 $\text{score}_{ij} = \mathbf{x}_i^T \mathbf{W}_Q \mathbf{W}_K^T \mathbf{x}_j / \sqrt{d_h}$ 。Q/K 联合定义一个 learned bilinear similarity。
2	V 决定被传输的内容	$\text{softmax}(\text{score})$ 只产生路由权重；真正被加权聚合的是 V。
3	“Q=问题、K=关键词、V=答案”只是直觉	训练不会强制固定语义角色；同一个 head 的功能还会随层、token 类型、上下文而变化。
4	scale 用 d_{head} ，不是 d_{model}	经典 scaled dot-product attention 除以 $\sqrt{d_k}$ ，通常 $d_k = d_{\text{head}}$ 。
5	RoPE 通常施加在 Q/K	它改变匹配几何；V 不参与位置相位匹配，因此主流 RoPE 架构通常不旋转 V。
6	GQA/MQA 的关键收益来自减少 K/V heads	decode 时瓶颈常是读取历史 KV cache 的内存带宽，而非生成当前 Q 的矩阵乘。
7	KV cache 可从投影形状直接计算	$\text{bytes} \approx 2 \times \text{layers} \times \text{seq} \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{bytes_per_elem}$ 。
8	Fused QKV 不是新算法	把三次 GEMM 合并为一次大 GEMM 后再 split，主要减少 kernel launch/内存流量。
9	QK-Norm 的目的不是“让向量更好看”	它控制 dot-product logits 的尺度，降低 softmax 过早饱和与训练不稳定风险。
10	FlashAttention 不改变 QKV 语义	它重排 tile/online-softmax 的执行，避免显式物化完整 $L \times L$ score matrix。
11	MLA 改写了“缓存什么”	通过低秩 latent 表示压缩 K/V 的长期状态，代表 attention 从 head-sharing 走向 latent compression。
12	三投影不是不可触碰的定律	2025-2026 有工作系统研究 Q/K/V 共享或消除；但目前属于新兴方向，不应把小规模结果直接当成大模型默认结论。

2. Q/K/V 位于 Transformer 的哪里

对 decoder-only Transformer, 一个典型 pre-norm attention 子层可以写成:

$$\begin{aligned}
 \mathbf{H} &= \text{RMSNorm}(\mathbf{X}) \\
 \mathbf{Q} &= \mathbf{H} \mathbf{W}_Q, \quad \mathbf{K} = \mathbf{H} \mathbf{W}_K, \quad \mathbf{V} = \mathbf{H} \mathbf{W}_V \\
 \mathbf{Q}\sim, \mathbf{K}\sim &= \text{PositionTransform}(\mathbf{Q}, \mathbf{K}) \quad \# \text{ e.g. RoPE} \\
 \mathbf{A} &= \text{softmax}((\mathbf{Q}\sim \mathbf{K}\sim^T) / \text{sqrt}(d_h) + \text{mask}) \\
 \mathbf{Y} &= \mathbf{A} \mathbf{V}, \quad \mathbf{O} = \mathbf{Y} \mathbf{W}_O, \quad \mathbf{X}_{\text{next}} = \mathbf{X} + \mathbf{O}
 \end{aligned}$$

因此 QKV projection 位于 residual stream 与 attention operator 的接口处。它不是“attention 之后的细节”，而是决定 attention 看见什么坐标系的入口。



The three projections are learned interfaces between the residual stream and the attention operator.

2.1 从 residual stream 的视角

- Residual stream 是跨层累计的信息载体。W_Q/W_K/W_V 每层重新读取这个公共状态，但读取目的不同。
- W_Q: 把当前 token 映射成“本 head 当前需要匹配什么”的坐标。
- W_K: 把可被访问 token 映射成“我怎样被匹配”的索引坐标。
- W_V: 把 token 映射成“被选中后要传输什么”的内容坐标。

Gotcha 不要把 Q/K/V 当成三个互不相关的语义 embedding。它们是同一 hidden state 的三个任务特定线性视图，而且 Q/K 的作用只有进入匹配函数后才完整定义。

3. 基：三次性投影究竟做了什

3.1 单 head 的双线性视角

设 token i 、 j 的 hidden state 分别为 x_i, x_j 。

$$\begin{aligned} q_i &= x_i W_Q, \quad k_j = x_j W_K \\ q_i k_j^T &= x_i (W_Q W_K^T) x_j^T \end{aligned}$$

令 $M = W_Q W_K^T$ ，则 score 本质上是 learned bilinear form。一个 head 并不是在原始 hidden space 中直接做 cosine similarity，而是在训练得到的匹配几何中比较两个 token。

Know-why Q 与 K 为什么可以不同？因为“发起查询的特征”与“被索引的特征”不必对称。若强制 $Q=K$ ，匹配矩阵受到更强的对称/PSD 类约束，可能限制表达。

3.2 Value 路径与 score 路径是两条不同通道

$$y_i = \sum_j \alpha_{ij} v_j, \quad \alpha_{ij} = \text{softmax}_j(\text{score}_{ij})$$

α 决定路由， v_j 决定被路由的内容。这种“寻址/内容分离”与数据库中的 key-value 思想相似，但这里 key/value 都是连续向量且共同训练。

3.3 Rank 视角

若每个 head 的维度为 d_h ，则 $M = W_Q W_K^T$ 的秩最多为 d_h 。这意味着 multi-head attention 用多个较低维双线性匹配器并行工作，再经 W_O 融合。

推论 “增加 head 数”并不只是增加并行注意力图数量，它也改变了每个匹配器的维数、总投影参数组织方式、KV cache 结构以及硬件映射。

4. Know-why : 为什么要区分 Query / Key / Value

设计	会发生什么
如果只有 $X \cdot X^T$	没有 learned projection, 匹配只能依赖 residual space 的原始几何, 所有层/heads 缺少专用检索坐标。
如果 $Q=K$	匹配更对称; 某些任务可能足够, 但会减少“查询角色”和“索引角色”的非对称自由度。
如果 $K=V$	索引表示与传输内容绑定; 可能节省参数/cache, 但“容易被找到”与“找到后传什么”不能独立优化。
如果 $Q=K=V$	注意力接近对单一表示的自匹配与聚合; 极简但表达约束最大。
三者独立	最大化寻址与内容通道自由度, 是经典 Transformer 的稳健默认。

值得注意的是, 独立三投影是工程上长期验证的默认, 并非数学上唯一可能。2025-2026 的新研究开始研究投影共享、固定某一投影或重参数化的等价性; 这些结果说明 QKV 可能存在冗余, 但大多仍受模型规模、归一化、位置编码和训练设置限制。

研究边界 “W_Q 在简化条件下可被吸收”不等于“所有现代 LLM 都应删除 W_Q”。RoPE、QK-Norm、head sharing、bias、低秩结构与优化动力学都会破坏简单的线性等价关系。

5. Shape、参数量、FLOPs 与初始化

5.1 标准 MHA shape

```
X: [B, L, d_model]
W_QKV: [d_model, 3 * h * d_head] # fused view
Q/K/V: [B, h, L, d_head]
Scores: [B, h, L, L]
Output: [B, L, d_model]
```

经典配置常有 $h * d_head = d_model$ ，因此 Q/K/V 三个投影各约 d_model^2 参数，W_O 再约 d_model^2 ，总 attention linear 参数约 $4 d_model^2$ （忽略 bias）。

5.2 GQA shape

```
Q: [B, h_q, L, d_head]
K: [B, h_kv, L, d_head]
V: [B, h_kv, L, d_head]
repeat/group mapping: each KV head serves h_q / h_kv query heads
```

GQA 下 W_Q 仍可接近 $d_model \times d_model$ ，但 W_K/W_V 的输出宽度缩小到 $h_kv * d_head$ 。

5.3 参数量近似

$$P_{QKV} = d_model * d_head * (h_q + 2 h_kv)$$

$$P_O = (h_q * d_head) * d_model$$

当 $h_q * d_head = d_model$ 时，减少 h_kv 会同时减少参数和 KV cache，但 GQA 的主要生产收益通常来自 decode 阶段历史 KV 读取量下降。

Gotcha 不要只用“参数量减少多少”解释 GQA。decode 是逐 token 的，历史 KV cache 每步都要参与 attention；内存带宽常比当前 token 的 QKV GEMM 更关键。

6. Multi-Head Attention : head 不是简单切片

原始 Transformer 将多个 attention heads 并行，每个 head 使用自己的 Q/K/V projection，再 concat 并通过 W_O 。该设计让模型在不同低维子空间中学习不同匹配关系。

$$\text{head}_r = \text{Attention}(X W_Q^r, X W_K^r, X W_V^r)$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

6.1 “先投影再切 head” vs “每个 head 三个小矩阵”

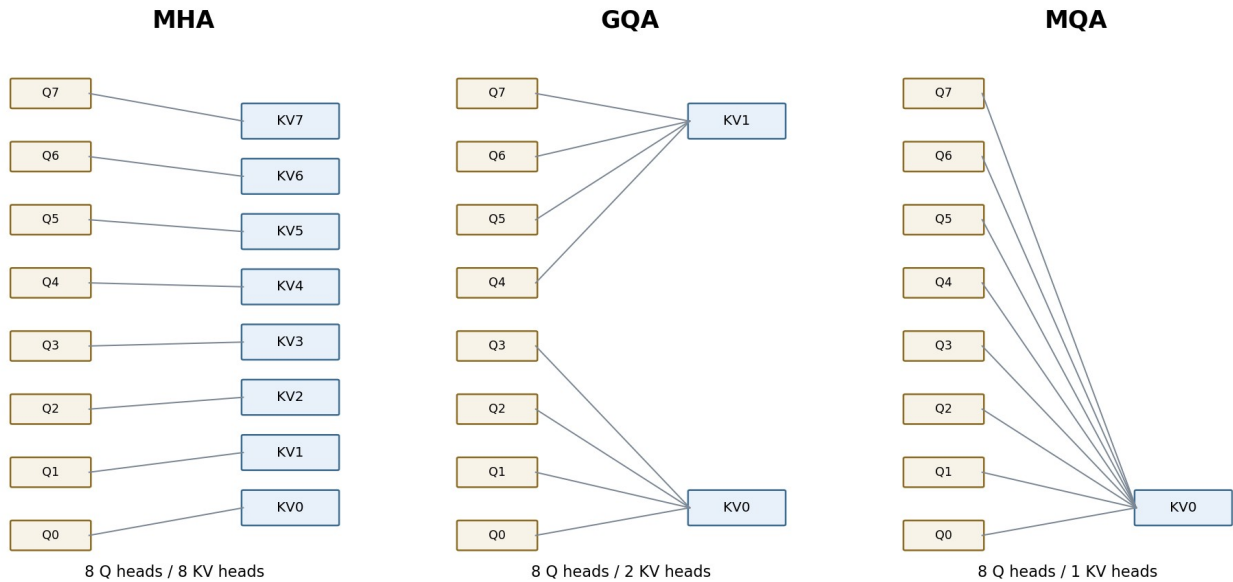
数学上可视为多个小矩阵；实现上通常把所有 heads 的投影拼成一个大矩阵，执行一个 GEMM 后 reshape/split。这样更容易利用 GPU 大矩阵乘吞吐。

6.2 Head dimension 的真正作用

- 决定单个 Q·K 点积的维度与 scaling 因子。
- 限制单 head 双线性匹配矩阵的 rank。
- 直接影响 RoPE 频率维度数、KV cache 每 head 宽度、FlashAttention kernel tile 选择。
- 不是越大越好；head 数与 head_dim 是表达、吞吐和 cache 的联合设计变量。

Gotcha d_{model} 必须能否整除 num_heads 取决于实现，不是数学定理。很多高性能 kernel 假设固定/常见 head_dim（如 64/128/256）以获得更好向量化。

7. MHA → MQA → GQA : KV 共享什么能加速



MQA (2019) 让所有 query heads 共享一组 K/V heads，以降低增量解码时反复读取 K/V 的内存带宽；GQA (2023) 在 MHA 与 MQA 之间取中间点，用若干 KV heads 服务更多 Q heads。

7.1 三种结构对比

结构	Q heads	KV heads	优点	典型风险
MHA	h	h	表达自由度高	KV cache 最大
MQA	h	1	decode 带宽最省	可能有质量损失/容量瓶颈
GQA	h	$g, 1 < g < h$	质量与速度折中	group 映射、kernel 支持更复杂

7.2 为什么共享 K/V 而不是共享 Q

自回归 decode 时，每一步只产生当前 token 的 Q/K/V，但必须读取全部历史 K/V。Q 只属于当前 query，不需要跨时间缓存。因此减少 K/V heads 直接缩小历史状态；共享 Q 不会获得同等 cache 收益。

Know-why MQA/GQA 是“让历史状态更便宜”，不是“让 attention 数学复杂度从 $O(L^2)$ 变成 $O(L)$ ”。prefill 仍要处理大量 token-token 交互；decode 的主要收益来自 KV-cache bytes 和 bandwidth。

8. Cross-Attention : Q 与 K/V 来自不同信息源

在 encoder-decoder 或多模态 cross-attention 中, Q/K/V 分离变得更直观:

$$\begin{aligned} \mathbf{Q} &= \mathbf{X_decoder} \mathbf{W_Q} \\ \mathbf{K} &= \mathbf{H_encoder} \mathbf{W_K}, \quad \mathbf{V} = \mathbf{H_encoder} \mathbf{W_V} \end{aligned}$$

Query 表示“当前 decoder 状态需要什么”; Key/Value 表示“encoder memory 如何被索引、被读取”。这也是 key-value memory 直觉最自然的场景。

8.1 Shape 可以不同

Q length = L_q
K/V length = L_{kv}
Scores = [B, heads, L_q , L_{kv}]

因此 PyTorch 等实现允许 kdim/vdim 与 embed_dim 不同; 但高性能 fused kernel 常对 shape 有额外限制。

8.2 多模态 Gotcha

- 图像/音频 encoder 输出尺度与文本 residual stream 不同, K/V projection 需要承担模态对齐。
- 若 QK normalization、RoPE 或 position id 仅按文本假设实现, cross-attention 可能出现 silent shape/position bug。
- 缓存 encoder K/V 时, 要区分静态 cross-KV 与逐 token 增长的 self-KV。

9. Position Encoding 与 Q/K : 为什么 RoPE 通常不作用于 V

RoPE 的核心是让 Q/K dot product 中出现与相对位置有关的旋转相位，因此它天然属于“匹配路径”。

$$\begin{aligned} \mathbf{q}_m &= \mathbf{R}_m \mathbf{q}, \quad \mathbf{k}_n = \mathbf{R}_n \mathbf{k} \\ \mathbf{q}_m^T \mathbf{k}_n &= \mathbf{q}^T \mathbf{R}_{(n-m)} \mathbf{k} \end{aligned}$$

Value 被 attention 权重加权后传输，它不参与产生相似度 score，因此主流 RoPE Transformer 通常只旋转 Q/K。

9.1 正确操作顺序

```
H = norm(X)
Q,K,V = project(H)
Q,K = reshape_to_heads(Q,K)
Q,K = apply_rope(Q,K, position_ids)
A = sdpa(Q,K,V,mask)
```

9.2 常见 position bugs

- cache decode 时使用局部 index 0 而不是绝对/逻辑 position。
- chunked prefill 的第二块重新从 0 编 position。
- beam reorder 了 KV cache 却没有同步 position/cache metadata。
- GQA repeat K 后才做 RoPE，导致不必要计算或 layout 错误；通常对原始 KV heads 做位置变换更自然。

连接上一章 Position Embedding 决定“地址如何编码”；Q/K projection 决定“地址进入哪个检索空间”。两者是在 dot-product 处真正合流。

10. Attention Scaling 、Softmax Saturation 与 QK-Norm

10.1 为什么除以 $\sqrt{d_{\text{head}}}$

若 q 、 k 各维近似零均值、单位方差， d_{head} 维点积的方差随 d_{head} 增长。经典 Transformer 用 $1/\sqrt{d_k}$ 控制 logits 尺度，避免 softmax 进入过度尖锐区域。

$$\text{score} = (Q K^T) / \sqrt{d_{\text{head}}}$$

10.2 Softmax saturation

当某些 logits 绝对值非常大，attention 分布趋近 one-hot；这不一定总是坏事，但若过早或由范数漂移导致，会产生梯度不稳定、head collapse 或数值溢出风险。

10.3 QK-Norm

QKNorm (2020) 直接归一化 query/key，并使用可学习尺度；后续大型模型/训练工作也采用 QK normalization 作为稳定化手段。OLMo 2 的消融报告 QK-Norm 与其 normalization 调整共同降低训练 spike 指标。

$$\mathbf{q_hat} = \mathbf{q} / \|\mathbf{q}\|, \quad \mathbf{k_hat} = \mathbf{k} / \|\mathbf{k}\|$$

$$\text{score} = \mathbf{g} * \mathbf{q_hat} \mathbf{k_hat}^T$$

Gotcha QK-Norm 的具体位置与形式不统一：L2Norm、LayerNorm、RMSNorm、per-head scale、是否在 RoPE 前后，都可能不同。迁移 checkpoint 时不能只看 “use_qk_norm=True”。

11. Fused QKV、Bias 与工程布局

11.1 Fused projection

实现常把 W_Q 、 W_K 、 W_V 拼成一个大矩阵 W_{QKV} :

$$[Q \mid K \mid V] = X W_{QKV}$$

这通常减少 kernel launch、改善 GEMM 尺寸和内存访问，但数学上仍是三组参数。

11.2 GQA 下切分更容易

```
q_width  = h_q  * d_head
kv_width = h_kv * d_head
fused_width = q_width + 2*kv_width
# split sizes must NOT assume 3 equal chunks
```

高频 Bug 把 GQA 的 fused QKV 输出按 1/3、1/3、1/3 等分，会产生 shape 正常但语义完全错的 silent bug。正确 split 必须按 $[h_q*d_{head}, h_{kv}*d_{head}, h_{kv}*d_{head}]$ 。

11.3 Bias

是否给 Q/K/V projection 加 bias 是架构选择。很多 decoder LLM 使用无 bias 线性层，但 BERT/原始实现家族常见 bias。加载权重、做量化或 fuse kernel 时必须严格匹配。

12. 训练动力学：梯度、范数、熵与 head collapse

Q/K 通过 score 共同影响 softmax 权重，因此它们的梯度高度耦合；V 的梯度更多通过加权聚合路径传播。

12.1 需要监控什么

指标	异常信号	可能原因
$\ Q\ / \ K\ $ per head	持续增长或极端 outlier	logit scale 漂移、LR/optimizer、缺少 QK-Norm
max attention logit	快速上升	Q/K norm 增长、scale 错误、精度问题
attention entropy	长期接近 0	过度尖锐、head collapse、mask/position bug
attention entropy	长期接近 $\log(L)$	近似均匀、未学到检索结构、初始化/梯度问题
Q/K/V grad norm	某一支异常小/大	fuse split、detach、量化、并行切分错误
head similarity	heads 高度相同	容量冗余、训练早期 collapse、共享设置过强

12.2 “尖锐 attention”不自等于好

一些 head 的确需要近似 hard routing，但若所有 head/层都趋近低熵，通常会牺牲组合信息并增加训练敏感性。评价必须结合任务、层深与 head 角色。

13. KV Cache : 从 projection 直接推导推理显存

$$\text{KV_bytes} = 2 * B * L * N_layers * h_kv * d_head * \text{bytes_per_element}$$

“2” 来自 K 和 V。Q 不缓存，因为 decode 时只需要当前 query。

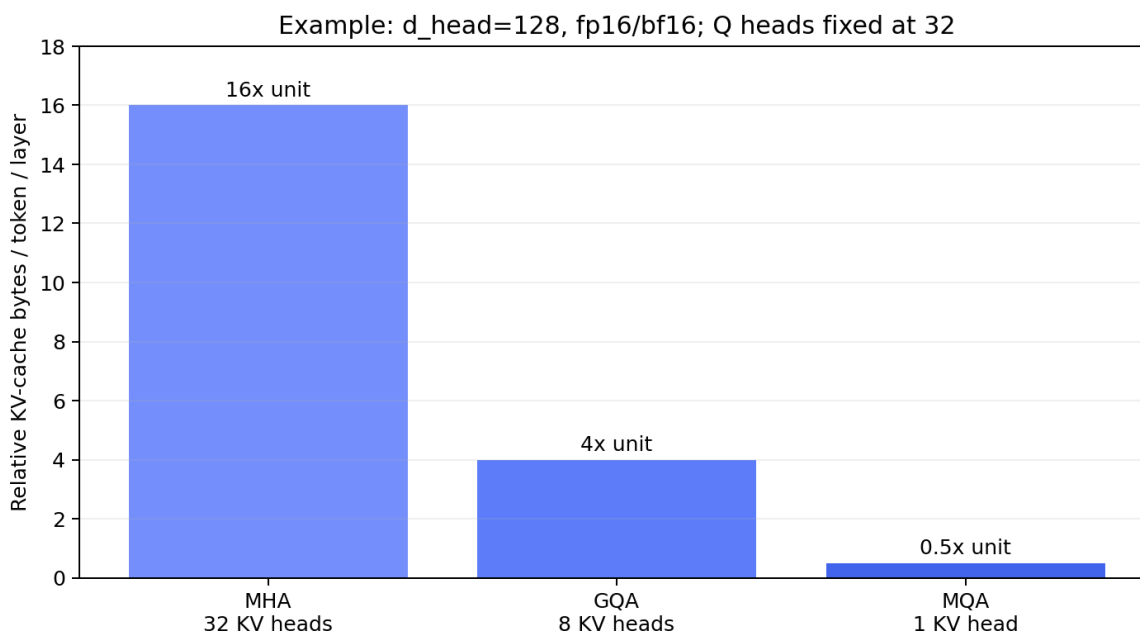
13.1 一个直观例子

假设 32 层、32 个 Q heads、8 个 KV heads、d_head=128、BF16/FP16=2 bytes、单序列 128K:

$$\text{per-token-per-layer} = 2 * 8 * 128 * 2 = 4096 \text{ bytes} = 4 \text{ KB}$$

$$\text{total} \approx 4 \text{ KB} * 32 * 131072 \approx 16 \text{ GB}$$

若改成 32 KV heads 的 MHA，其他条件不变，约为 4 倍，即约 64 GB。这里尚未计入 allocator/page metadata、padding、alignment 等系统开销。



Know-why 长上下文推理中，KV head 数是硬件成本的一等参数。模型架构里的一个“小整数”会被 $\text{seq_len} \times \text{layers} \times \text{batch}$ 放大成 GB 级差异。

14. Tensor Parallel / Sequence Parallel / Context Parallel

14.1 常见 Tensor Parallel 切法

大模型训练通常让 QKV projection 使用 column-parallel: 按输出维切 W_QKV, 各 GPU 持有一部分 heads; W_O 再用 row-parallel 汇总。

```
QKV: ColumnParallelLinear(d_model -> qkv_width)
    each rank owns subset of Q/K/V channels or heads
Attention: local heads
W_O: RowParallelLinear(heads*d_head -> d_model)
```

14.2 GQA 的 divisibility Gotcha

- h_q 必须与 TP degree 的 head partition 兼容。
- h_{kv} 少于 TP ranks 时, KV heads 可能需要 replicate, 而非均匀 shard。
- 不同框架对 repeat_kv 的时机不同: 通信前、通信后或 kernel 内广播。
- checkpoint weight layout 可能是 [Q,K,V] contiguous, 也可能按 head interleave; 不能仅凭 tensor shape 猜。

14.3 Context Parallel

Context parallel 会按序列维分 token, 但 attention 需要跨分片访问 K/V, 因此 QKV 布局与 all-gather/ring attention 通信强相关。长上下文架构设计必须同时考虑“缓存大小”和“跨设备搬运量”。

15. FlashAttention 与 QKV : 算法相同、执行路径不同

FlashAttention (2022) 通过 IO-aware tiling 与 online softmax, 减少 HBM $\leftrightarrow$ SRAM 数据搬运并避免把完整 $L \times L$ attention matrix 写回 HBM; FlashAttention-2 进一步改善并行与 work partition。

15.1 它没有改什么

- Q/K/V 的数学含义不变。
- scaled dot-product attention 的结果在数值容差内仍是 exact attention。
- GQA/MQA 的 head-sharing 仍由输入 shape 决定。

15.2 它改变什么

- kernel 可能要求 Q/K/V 连续布局、特定 dtype/head_dim。
- mask、causal、dropout、window attention 等功能必须走支持的 kernel path。
- 某些 GQA 实现通过 enable_gqa 或内部 broadcast, 而不是显式 repeat K/V; 显式 repeat 会浪费显存。

PyTorch 现状 当前 PyTorch scaled_dot_product_attention 会根据设备、dtype、shape 等选择可用实现, 并支持 GQA 相关路径; 生产代码应把“数学 correctness test”和“具体 backend performance test”分开。

16. Quantization / FP8 / Low Precision 的 QKV 风险

16.1 Q/K 比普通线性层更敏感的原因

Q/K 误差会先进入乘法形成 score，再经过 softmax 的指数归一化，可能放大排序变化；V 误差则直接进入被加权的内容。二者的误差传播机制不同。

16.2 实践检查

- 分别统计 $W_Q/W_K/W_V$ 与 activation Q/K/V 的 max、RMS、outlier percentile。
- 不要只看最终 perplexity；同时比较 attention logits、entropy、top-k attention index 的一致性。
- 长上下文下误差更容易积累并改变极值竞争，必须在目标 context length 上验证。
- KV cache quantization 需要单独评估：它影响所有后续 decode step，而不是一次性线性层误差。

Gotcha “QKV 都是 Linear，所以统一量化策略即可” 是危险假设。权重分布、activation outlier、score sensitivity 和 cache 生命周期不同。

17. MLA : 从显式 K/V 到 latent KV

DeepSeek-V2 引入 Multi-head Latent Attention (MLA)，核心目标是进一步压缩 decode 所需 KV cache。与 GQA 通过“减少 KV heads”不同，MLA 通过低秩联合压缩，把历史 token 的 key/value 信息缓存为更低维 latent，再在计算路径中恢复/吸收投影。

17.1 演化逻辑

```

MHA: cache per-head K and V
  ↓ share heads
GQA: cache fewer K/V heads
  ↓ one KV head
MQA: cache one K/V head
  ↓ compress representation itself
MLA: cache low-dimensional latent state
  
```

17.2 为什么它属于 QKV projection 主题

MLA 的核心创新正是重新因式分解 K/V projection 与 cache：哪些矩阵可以在训练参数里保留、哪些变换可以在 decode 时“吸收到 query 侧”、哪些状态必须逐 token 保存。

17.3 2026 的新问题：QK-Norm × MLA

2026 的 QK-Normed MLA 工作指出，表面上 post-projection QK RMSNorm 似乎迫使系统缓存完整 K，但可以通过把静态权重吸收到 query-side projection，并仅缓存必要的动态统计，保留 latent decode path。这说明 normalization 与 cache factorization 必须联合设计。

趋势判断 QKV 的未来优化越来越不像“改一个线性层”，而是围绕 representation factorization、cache、position、normalization 和硬件通信做协同设计。

18. 45 个 Gotchas : QKV 最容易踩的坑

类别	Gotcha
Shape	把 d_model 当成 d_head 做 scaling。
Shape	GQA fused QKV 误按三等分 split。
Shape	reshape 后忘记 transpose, L 与 head 维交换错误。
Shape	view 用在 non-contiguous tensor 上导致错误/隐式复制。
Mask	boolean mask 与 additive -inf mask 语义混淆。
Mask	causal mask 方向反了, 模型偷看未来。
Mask	padding mask broadcast 到错误 head/sequence 维。
RoPE	在 V 上错误施加 RoPE。
RoPE	decode position 从 0 重启。
RoPE	partial rotary 时旋转维数与 head_dim 不匹配。
RoPE	RoPE 前后 QK-Norm 顺序与 checkpoint 不一致。
GQA	显式 repeat_interleave K/V 后写入 cache, 白白放大 cache。
GQA	h_q/h_kv grouping mapping 错位。
GQA	TP ranks 多于 KV heads 时未正确 replicate。
Cache	K/V cache layout [B,H,L,D] 与 kernel 期望 [B,L,H,D] 不一致。
Cache	prefill 与 decode 使用不同 dtype/scale。
Cache	beam reorder 只重排 tokens 没重排 KV。
Cache	sliding window 淘汰 K/V 后 position metadata 不同步。
Numerics	scale 在 softmax 后才乘。
Numerics	FP16 中手写 exp/sum, 未使用稳定 softmax。
Numerics	Q/K norm outlier 没有监控。
Numerics	attention logits 无限制增长却只看 loss。
Training	QKV 权重初始化方差与残差/深度策略不匹配。
Training	QK-Norm 参数被 weight decay 处理方式不当。
Training	fused QKV 中某一 slice 不参与 optimizer due to parameter wrapping bug。
Training	gradient checkpointing 重算路径与 cache path 混用。
Bias	checkpoint 有 qkv_bias, 本地实现设 bias=False。
Bias	bias 顺序/切分与 fused weight layout 不一致。
Quant	Q/K/V 使用同一 per-tensor scale 且被极端 outlier 主导。
Quant	KV cache 量化只测短上下文。
Quant	量化前后 top-attended token 排名大幅变化未发现。
Parallel	checkpoint head order 与 TP shard order 不一致。
Parallel	QKV weight concat 顺序 QKV vs QVK/KQV 搞错。
Parallel	all-gather 重复了本来可本地计算的 KV heads。
Flash	为了适配 kernel 显式 materialize $L \times L$ mask, 抵消 FlashAttention 收益。
Flash	unsupported head_dim 静默 fallback 到慢 kernel。
Flash	dropout 在 eval 未关闭或 train parity 测试没固定随机性。
Interpret	把某个 head 固定命名为“语法 head”, 过度解释。

Interpret	把 attention weight 当成完整因果归因。
Interpret	认为 Q/K/V 各维具有稳定可解释语义。
Cross	cross-attention 把 decoder hidden 同时用于 K/V。
Cross	static encoder KV 每 decode step 重新计算。
Testing	只测试 batch=1、L=短序列，没覆盖 ragged/padding。
Testing	只比较输出，不比较梯度 parity。
Testing	prefill 输出正确但 decode-with-cache 不一致。

19. Know-how : 从零实现与生产 Checklist

19.1 从零实现顺序

1. 先实现 single-head、无 RoPE、无 cache 的 reference attention，并用 float64/float32 小 tensor 做单元测试。
2. 扩展到 multi-head，只做 reshape/transpose，不先做 GQA。
3. 加入 causal/padding mask，验证每一行 softmax 权重和为 1，未来位置为 0。
4. 加入 RoPE，仅作用 Q/K；用同样 position 的相对性质做测试。
5. 加入 KV cache，分别验证 full-forward 与 token-by-token decode 的 logits parity。
6. 再加入 GQA/MQA，先使用显式 repeat 作为 correctness reference，再替换成 fused/kernel broadcast。
7. 最后接 SDPA/FlashAttention，并保留 reference implementation 作为回归基线。

19.2 训练前 Checklist

- 记录 d_model/h_q/h_kv/d_head/rope_dim/scale 的配置快照。
- 确认 QKV bias、QK norm 类型与顺序。
- 用固定 seed 记录 Q/K/V RMS、attention-logit max、entropy 的初始化基线。
- 做 forward+backward parity，尤其检查 fused QKV 各 slice 的 grad。
- 在目标精度 BF16/FP8 下做数值 smoke test。

19.3 推理前 Checklist

- 计算理论 KV cache bytes，并与实际 allocator 监控对账。
- 分别 benchmark prefill tokens/s 与 decode tokens/s。
- 检查 cache layout、page size、GQA broadcast 是否 materialize。
- 做 1/2/4/8 batch 与短/长 context 梯度式性能扫描。
- 做 chunked prefill / beam / speculative / sliding-window parity。

20. Know-why : 因果原理总表

问题	核心因果
为什么 Q/K 要投影?	让每层/head 学自己的匹配几何, 而非被 residual-space 原始距离限制。
为什么 V 要独立?	把“被找到的条件”和“找到后传输的内容”解耦。
为什么 scale 是 $\sqrt{d_{\text{head}}}$?	控制 dot-product 方差随维度增长。
为什么 Q 不进 KV cache?	历史 query 不再被未来 token 使用; 未来 query 只需匹配历史 K/V。
为什么 MQA/GQA 快?	减少每步 decode 需要从显存读出的历史 K/V bytes。
为什么 RoPE 改 Q/K 不改 V?	位置要改变匹配关系; V 只承载被路由内容。
为什么 QK-Norm 能稳训练?	限制 query/key 范数对 logits 尺度的无界放大。
为什么 fused QKV 更快?	把多个小 GEMM/kernel launch 合成更高算术强度的大 GEMM。
为什么 FlashAttention 不需要 $L \times L$ HBM matrix?	利用 tile + online softmax 在 on-chip memory 中累积正确归一化结果。
为什么 GQA 不是免费午餐?	共享 K/V 降低 memory capacity, 自由度与吞吐之间存在 trade-off。
为什么 MLA 更进一步?	它不只减少 heads, 而是降低每 token 必须长期保存的状态维度。
为什么 QKV 配置是硬件参数?	head 数、head_dim、cache dtype 被 $\text{layers} \times \text{seq} \times \text{batch}$ 放大并决定通信/带宽。

21. Debugging / Monitoring 指标

21.1 每层每 head 建议监控

Metric	建议统计	用途
Q/K/V RMS	mean, p95, max	发现范数漂移/outlier
attention logits	mean/std/max/min	监控 softmax saturation
attention entropy	mean/p10/p90	发现 collapse 或过度均匀
top-1 attention distance	距离分布	检查 position / mask 行为
QK cosine / dot	分布	检查 QK geometry
grad norm W_Q/W_K/W_V	per-layer	定位训练不平衡
update/weight ratio	per matrix	检查 optimizer scale
cache bytes	theory vs actual	发现 materialization/replication
prefill/decode parity	max abs / rel error	发现 cache/position bugs

21.2 最小告警逻辑

if max_attn_logit grows monotonically for many steps: inspect Q/K norm and LR
if entropy -> 0 across most heads: inspect scale, QK norm, mask
if decode logits != full_forward logits: inspect cache order + position_ids + RoPE
if actual KV bytes >> theoretical: inspect repeat_kv / dtype / page padding / replication

22. Ablation Matrix : 如何验证设计选择

QKV 的设计不能只用单次 loss 判断。建议固定 token budget、data order、optimizer 与总参数规模，做结构化消融。

变量	候选	至少观察
KV heads	MHA / GQA-8 / GQA-4 / MQA	val loss, long-context, decode tok/s, cache GB
QK normalization	none / RMSNorm / L2 QKNorm	loss spikes, max logits, entropy, downstream
QKV bias	on/off	loss, convergence, quantization behavior
head_dim	64/128/256	quality, kernel efficiency, RoPE behavior
KV cache dtype	BF16/FP8/int8	ppl, long-context, generation parity
projection sharing	none / K=V / Q=K 等	质量、参数、稳定性（研究性）
RoPE order	norm→RoPE vs RoPE→norm	checkpoint compatibility + long context
kernel	reference / SDPA / Flash	parity + prefill/decode speed

Know-how 性能实验必须把 prefill 与 decode 分开；质量实验必须覆盖短上下文与目标最长上下文。否则很容易选择一个“短 benchmark 很快、长上下文反而更贵/更差”的配置。

23. PyTorch MVP 与 Parity Tests

23.1 最小 GQA 投影

```
class QKVProj(nn.Module):
    def __init__(self, d_model, n_q, n_kv, d_head, bias=False):
        super().__init__()
        self.n_q, self.n_kv, self.d_head = n_q, n_kv, d_head
        out = (n_q + 2*n_kv) * d_head
        self.qkv = nn.Linear(d_model, out, bias=bias)

    def forward(self, x):
        z = self.qkv(x)
        qn = self.n_q * self.d_head
        kn = self.n_kv * self.d_head
        q, k, v = z.split([qn, kn, kn], dim=-1)
        B, L, _ = x.shape
        q = q.view(B, L, self.n_q, self.d_head).transpose(1, 2)
        k = k.view(B, L, self.n_kv, self.d_head).transpose(1, 2)
        v = v.view(B, L, self.n_kv, self.d_head).transpose(1, 2)
        return q, k, v
```

23.2 Reference GQA repeat (只用于 correctness baseline)

```
groups = n_q // n_kv
k_ref = k.repeat_interleave(groups, dim=1)
v_ref = v.repeat_interleave(groups, dim=1)
y_ref = F.scaled_dot_product_attention(q, k_ref, v_ref, is_causal=True)
```

23.3 必做 Parity Tests

- fused QKV vs 三个独立 Linear: forward/grad 对齐。
- full sequence vs token-by-token KV cache: 每个位置 logits 对齐。
- reference repeat GQA vs kernel-native GQA: 输出对齐。
- RoPE cache offset=0/1/chunk: 逻辑 position 对齐。
- BF16 与 FP32 reference: 记录误差随 sequence length 的增长。

24. 2025-2026 新研究与开放问题

24.1 “三投影真的都必要吗？”

2025 的工作从理论与小规模实验讨论 Q/K/V triplet 的冗余；2026 也出现系统比较共享约束的研究。它们提出一个值得重视的问题：经典 QKV 参数化可能存在可吸收的线性自由度。

但成熟结论仍应谨慎：当架构含 RoPE、QK-Norm、GQA/MLA、非标准 scaling、低精度与超大规模训练时，简化参数化的等价性和优化可达性并不自动成立。

24.2 QK-Norm 与 latent attention 的兼容

QK-Normed MLA（2026）显示，norm 与 latent cache 的冲突可通过代数重参数化解决。这类工作体现了未来方向：把“数学等价变换”用于减少缓存/通信，而不是只减少 FLOPs。

24.3 Cost-optimal GQA

2025 的长上下文研究指出，常用 GQA head 配置未必对目标 context/hardware 成本最优。设计空间应联合考虑 h_q 、 h_{kv} 、 d_{head} 、模型宽度、序列长度与硬件成本。

24.4 K=V / value reuse

2026 的一些新模型与研究探索复用 key/value 或更强共享，以进一步减少 KV 状态。此方向值得跟踪，但需要特别关注质量、归一化和位置编码的耦合。

研究方法 对这些新方案，优先相信“同 token budget + 同参数/算力约束 + 多尺度模型 + 长上下文 + 真实硬件”的受控实验，而不是只看参数减少比例。

25. 面试与系统设计题

1. 为什么 Q/K 可以看成 learned bilinear metric? 推导 $\mathbf{x}_i^T \mathbf{W}_Q \mathbf{W}_K^T \mathbf{x}_j$ 。
2. 为什么 Value 不需要参与 attention score? 如果 $K=V$ 会发生什么约束?
3. 为什么 scaled dot-product 用 $\sqrt{d_{\text{head}}}$? 如果漏掉会发生什么?
4. MHA、GQA、MQA 的参数数量和 KV cache 分别如何计算?
5. 给定 48 层、64 Q heads、8 KV heads、 $\text{head_dim}=128$ 、BF16、context=256K, 估算单序列 KV cache。
6. 为什么自回归 decode 不缓存 Q?
7. RoPE 为什么通常只应用 Q/K?
8. QK-Norm 解决什么问题? 它与 $1/\sqrt{d_{\text{head}}}$ 是替代还是可以共存?
9. 为什么 fused QKV 在 GQA 下不能三等分?
10. 如何证明你的 KV cache 实现没有 position offset bug?
11. 为什么 FlashAttention 是 exact attention, 但显存明显降低?
12. 如果 GQA 从 32 KV heads 降到 8, decode 为什么可能接近 4 倍降低 KV bandwidth, 但整体 tokens/s 不一定 4 倍?
13. Tensor Parallel 中 KV heads 少于 GPU 数时怎么办?
14. 量化 Q/K 和量化 V 的误差传播路径有什么不同?
15. MLA 与 GQA 的核心差异是什么?
16. 如果训练中 max attention logit 持续增长, 你会按什么顺序排查?
17. 为什么 attention entropy 过低/过高都值得关注?
18. 如果 full-forward 正确但 cached decode 错, 最可能的五类原因是什么?
19. “attention weights = feature importance” 为什么不严格成立?
20. 你会如何做 QKV projection sharing 的受控 ablation?

26. 参考文献与来源分级

本报告优先使用原始论文、技术报告和官方框架文档。2025-2026 的新结论单独标为“新兴研究”，避免与长期验证的标准设计混淆。

- [S1] Vaswani et al. Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.
- [S2] Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. 2019. arXiv:1911.02150.
- [S3] Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. 2023. arXiv:2305.13245.
- [S4] Henry et al. Query-Key Normalization for Transformers. 2020. arXiv:2010.04245.
- [S5] Dehghani et al. Scaling Vision Transformers to 22 Billion Parameters. 2023. arXiv:2302.05442.
- [S6] Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. arXiv:2205.14135.
- [S7] Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 2023. arXiv:2307.08691.
- [S8] Grattafiori et al. The Llama 3 Herd of Models. 2024. arXiv:2407.21783.
- [S9] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv:2405.04434.
- [S10] DeepSeek-AI. DeepSeek-V3 Technical Report. 2024. arXiv:2412.19437.
- [S11] OLMo Team. 2 OLMo 2 Furious. 2024/2025. arXiv:2501.00656.
- [S12] PyTorch documentation: torch.nn.functional.scaled_dot_product_attention and torch.nn.MultiheadAttention (accessed 2026-08).
- [S13] Han et al. QK-Normed MLA: QK normalization without full key caching. 2026. arXiv:2606.16310.
- [S14] Karbevski et al. On the Necessity of the Query, Key, Value Weight Triplet in Attention. 2025. arXiv:2510.23912. (emerging)
- [S15] Do Transformers Need Three Projections? Systematic Study of QKV Projection Sharing. 2026. arXiv:2606.04032. (emerging)
- [S16] Cost-Optimal Grouped-Query Attention for Long-Context Transformers. 2025. arXiv:2503.09579. (emerging)

最终结论 Q/K/V projection 是 Transformer 中最值得从“公式层”提升到“系统层”理解的模块之一：QK 决定信息路由几何，V 决定内容通道；而 head sharing、normalization、RoPE、KV cache、FlashAttention、低精度和分布式并行都围绕这三条路径发生耦合。